ILIJA

Offenes Leuchten v5.0

Autonomer KI-Agent

BEDIENUNGSANLEITUNG

Vollständige Installations- und Bedienungsanleitung

inkl. manuelle Installation, Ordnerstruktur & Fehlerbehandlung

Eigenschaft	Information
Version	5.0
Betriebssystem	Ubuntu / Debian Linux (empfohlen), macOS
Python	3.10 oder höher
Letzte Aktualisierung	2026
Lizenz	MIT – freie Nutzung & Weitergabe

Inhaltsverzeichnis

KAPITEL 1

Über Ilija

Was ist dieser autonome KI-Agent?

1.1 Was ist Ilija?

Ilija ist ein autonomer KI-Agent 2013 kein einfacher Chatbot. Er denkt selbst, plant seine Schritte und kann sich neue F00e4higkeiten zur Laufzeit selbst programmieren. Ilija verkoerpert das Projekt Offenes Leuchten 2013 ein Open-Source-Experiment in autonomer KI.

1.2 Kernfunktionen

Langzeitgedächtnis

Ilija erinnert sich dauerhaft an alles was du ihm sagst – auch nach dem Neustart. Powered by ChromaDB (lokale Vektordatenbank).

Skill-System

Ilija kann sich selbst neue Python-Skills programmieren, sie speichern und sofort nutzen. Einfach beschreiben was du brauchst.

WhatsApp-Assistent

Überwacht WhatsApp Web automatisch, antwortet auf Nachrichten, vereinbart Termine und nimmt Nachrichten entgegen.

Telegram-Fernsteuerung

Steuere Ilija von überall per Telegram-Bot. Unterstützt Sprache, Bilder, Dateien und alle Befehle.

Web-Interface

Moderne Browser-Oberfläche mit Mikrofon-Aufnahme, Datei-Upload, Skill-Reload und mobilem Design.

Multi-Provider

Ilija wählt automatisch: Claude → ChatGPT → Gemini → Ollama (lokal, offline). Alle Provider kombinierbar.

1.3 Systemvoraussetzungen

Anforderung	Minimum	Empfohlen
Betriebssystem	Ubuntu 20.04 / Debian 11 / macOS 12	Ubuntu 22.04 LTS oder neuer
Python	3.10	3.11 oder 3.12
RAM	4 GB	8 GB oder mehr
Speicher	5 GB frei	20 GB (für Whisper/Ollama)
Internet	Für API-Calls	Stabile Verbindung

Anforderung	Minimum	Empfohlen
Browser (WhatsApp)	Google Chrome	Google Chrome (aktuell)

KAPITEL 2

Schnellstart

Installation mit install.sh (empfohlen)

2.1 Automatische Installation

Der einfachste Weg. Das Installationsskript führt dich durch alle Schritte interaktiv.

1. Projekt herunterladen oder von GitHub klonen:

```
git clone https://github.com/<user>/offenes-leuchten.git

cd offenes-leuchten
```

2. Skript ausführbar machen und starten:

```
chmod +x install.sh

./install.sh
```

2.2 Was das Skript macht

Das Installationsskript führt dich in 7 Schritten durch die komplette Einrichtung:

Schritt	Beschreibung
0 – Installationspfad	Wahl des Zielordners, optional Dateien dorthin kopieren
1 – Python prüfen	Version wird geprüft, mindestens 3.10 erforderlich
2 – Ollama (optional)	Lokales KI-Modell installieren und herunterladen
3 – Python-Pakete	Alle Abhängigkeiten werden automatisch installiert
4 – API-Keys	Cloud-Provider (Claude / ChatGPT / Gemini) konfigurieren
5 – Telegram-Bot	Bot erstellen und einrichten (mit Schritt-für-Schritt-Anleitung)
6 – Kennenlernen	Erklärung aller Funktionen und Beispiel-Befehle
7 – Starten	Wahl: Web-Interface / Telegram / Beide / Terminal

✅ Nach erfolgreicher Installation

Das Web-Interface ist erreichbar unter: http://localhost:5000
Im lokalen Netzwerk: http://<deine-IP>:5000
Telegram-Bot: Schreibe /start in Telegram
Terminal: python kernel.py starten

KAPITEL 3

Manuelle Installation

Schritt-für-Schritt ohne install.sh

Wenn du install.sh nicht verwenden möchtest oder kannst (z.B. auf einem Server ohne interaktive Shell), folge dieser vollständigen manuellen Anleitung.

3.1 Python-Umgebung einrichten

3. Wechsle in den Projektordner:

```
cd /pfad/zu/Ilija_full_evo2
```

4. Virtuelle Python-Umgebung erstellen:

```
python3 -m venv venv
```

5. Virtuelle Umgebung aktivieren:

```
source venv/bin/activate
```

(Windows: venv\Scripts\activate.bat)

6. pip aktualisieren:

```
pip install --upgrade pip
```

3.2 Pflichtpakete installieren

Diese Pakete werden von allen Kernkomponenten benötigt und müssen zwingend installiert werden:

```
pip install flask>=3.0.0
pip install flask-cors>=4.0.0
pip install python-dotenv>=1.0.0
pip install requests>=2.31.0
pip install anthropic>=0.40.0
pip install openai>=1.54.0
pip install ollama>=0.1.0
pip install beautifulsoup4>=4.12.0
pip install lxml>=4.9.0
pip install chromadb>=0.4.0
pip install sentence-transformers>=2.2.0
pip install python-telegram-bot>=20.0
pip install selenium>=4.0.0
pip install webdriver-manager>=4.0.0
```

⚠️ **Wichtig: beautifulsoup4 und lxml**
Diese beiden Pakete werden von webseiten_inhalt_lesen.py benötigt.
Ohne sie schlägt der Skill 'Webseiten lesen' beim Import fehl.
Beide müssen installiert sein – auch wenn du diesen Skill nicht aktiv nutzt.

3.3 Optionale Pakete

Paket	Für was	Installation
openai-whisper	Lokale Spracherkennung (WhatsApp, Telegram, Web)	pip install openai-whisper
PyPDF2	PDF-Dateien lesen (Web-Interface & Telegram)	pip install PyPDF2
pdfplumber	PDF-Extraktion (Alternativ zu PyPDF2)	pip install pdfplumber
python-docx	Word-Dokumente lesen (.docx, .doc)	pip install python-docx

3.4 Google Chrome installieren (für WhatsApp)

Der WhatsApp-Skill steuert WhatsApp Web über einen Chrome-Browser. Chrome muss auf dem System installiert sein:

```
# Ubuntu/Debian:
wget https://dl.google.com/linux/direct/google-chrome-stable_current_amd64.deb
sudo apt install ./google-chrome-stable_current_amd64.deb
rm google-chrome-stable_current_amd64.deb

# Version prüfen:
google-chrome --version
```

3.5 Ollama installieren (optionales lokales Modell)

```
# Ollama installieren:
curl -fsSL https://ollama.com/install.sh | sh

# Empfohlenes Modell herunterladen:
ollama pull qwen2.5:7b

# Verfügbare Modelle anzeigen:
ollama list
```

3.6 Gedächtnis-Modell vorladen

Beim ersten Start versucht Ilija das Sentence-Transformer-Modell herunterzuladen. Du kannst dies manuell vorbereiten:

```
python3 -c "
from sentence_transformers import SentenceTransformer
SentenceTransformer('all-MiniLM-L6-v2')
print('Modell bereit')"
```

KAPITEL 4

Konfiguration

Die .env-Datei einrichten

4.1 .env-Datei erstellen

7. Beispiel-Konfiguration kopieren:

```
cp .env.example .env
```

8. .env mit einem Texteditor öffnen:

```
nano .env
```

4.2 Alle Konfigurationsvariablen

Variable	Beschreibung	Pflicht?
ANTHROPIC_API_KEY	API-Key für Claude (Anthropic) – beste Qualität	Optional
OPENAI_API_KEY	API-Key für ChatGPT (OpenAI)	Optional
GOOGLE_API_KEY	API-Key für Gemini (Google) – kostenloses Kontingent	Optional
TELEGRAM_BOT_TOKEN	Token deines Telegram-Bots (von @BotFather)	Optional
TELEGRAM_ALLOWED_USERS	Deine Telegram User-ID (Sicherheits-Whitelist)	Optional
ANONYMIZED_TELEMETRY	ChromaDB-Telemetrie deaktivieren. Wert: False	Empfohlen

4.3 API-Keys beantragen

Provider	Webseite	Besonderheit
Claude (Anthropic)	console.anthropic.com	Kostenloser Testkredit bei Registrierung
ChatGPT (OpenAI)	platform.openai.com/api-keys	Kostenpflichtig, sehr hohe Qualität
Gemini (Google)	aistudio.google.com/app/apikey	Kostenloses Kontingent verfügbar

4.4 Telegram-Bot einrichten

Schritt-für-Schritt-Anleitung:

9. Öffne Telegram und suche nach @BotFather (offizieller Bot mit blauem Haken).
10. Schreibe: /newbot
11. Gib einen Anzeigenamen ein, z.B.: Ilija
12. Gib einen Username ein, z.B.: mein_ilija_bot (muss auf 'bot' enden)
13. Kopiere den Token, z.B.: 1234567890:AAHxxx...
14. Suche in Telegram nach @userinfobot und tippe /start – du erhältst deine User-ID.
15. Trage beides in die .env ein:

```
TELEGRAM_BOT_TOKEN=1234567890:AAHxxx...
TELEGRAM_ALLOWED_USERS=123456789
```

🔒 **Sicherheit**

Die .env-Datei enthält vertrauliche API-Keys – teile sie niemals!
Die .gitignore stellt sicher, dass .env nicht auf GitHub hochgeladen wird.
TELEGRAM_ALLOWED_USERS: Nur diese User-ID kann Ilija steuern.
Andere Telegram-Nutzer die den Bot finden erhalten keine Antwort.

KAPITEL 5

Ordnerstruktur

Damit das System korrekt funktioniert

5.1 Vollständige Ordnerstruktur

Das Projekt muss exakt diese Struktur haben. Fehlende Dateien oder falsch platzierte Komponenten führen zu Importfehlern.

```
Ilija_full_evo2/                    ← Projektroot (Arbeitsverzeichnis)
│
├── install.sh                      ← Installationsskript
├── web_server.py                   ← Flask Web-Interface (Start: python
web_server.py)
├── telegram_bot.py                 ← Telegram-Bot (Start: python telegram_bot.py)
├── kernel.py                       ← Zentraler Agent + Terminal-Modus
├── providers.py                    ← KI-Provider (Claude/GPT/Gemini/Ollama)
├── skill_manager.py                ← Dynamisches Skill-Laden & Ausführen
├── agent_state.py                  ← Zustandsautomat
├── autonomy_loop.py                ← Autonomie-Loop
├── skill_policy.py                 ← Sicherheits-Layer (SAFE/INTERACTIVE/RISKY)
├── skill_scoring.py                ← Skill-Bewertung & Statistiken
├── skill_versioning.py             ← Versionierung & automatische Backups
├── skill_validator.py              ← Skill-Code-Validierung
├── skill_registry.py               ← Geschützte Skills & Status-Verwaltung
├── model_registry.py               ← Dynamische Modell-Konfiguration
├── models_config.json              ← Aktuelle Modell-Einstellungen
├── system_config.py                ← System-Konfiguration
│
├── .env                            ← API-Keys (NICHT auf GitHub!)
├── .env.example                    ← Vorlage (auf GitHub)
├── .gitignore                      ← Was nicht auf GitHub kommt
├── requirements.txt                ← Python-Abhängigkeiten
├── skill_scores.json               ← Skill-Statistiken (auto-generiert)
│
├── skills/                         ← ALLE Skills müssen hier liegen
│   ├── __init__.py                 ← Pflicht! (leere Datei)
│   ├── basis_tools.py
│   ├── gedaechtnis.py
│   ├── skill_factory_improved.py
│   ├── whatsapp_autonomer_dialog.py
│   ├── whatsapp_lesen.py
│   ├── whatsapp_senden.py
│   ├── webseiten_inhalt_lesen.py
│   ├── browser_oeffnen.py
│   ├── datei_lesen.py
│   ├── cmd_ausfuehren.py
│   ├── ... (weitere Skills)
│   └── .skill_backups/             ← Automatische Backups bei Skill-Änderungen
│       └── .gitkeep
```

```
│
├── templates/                    ← Flask HTML-Templates
│   └── index.html                ← Web-Interface Template (PFLICHT)
│
├── static/                       ← Statische Web-Dateien
│   ├── app.js                    ← Frontend-Logik (PFLICHT)
│   └── style.css                 ← Design (PFLICHT)
│
├── memory/                       ← Langzeitgedächtnis (auto-erstellt)
│   ├── .gitkeep
│   └── ilija_db/                 ← ChromaDB-Datenbank (nach erstem Start)
│
├── venv/                         ← Virtuelle Python-Umgebung (nach Installation)
│
├── whatsapp_kalender.txt         ← Terminkalender (auto-erstellt)
├── whatsapp_nachrichten.txt      ← Hinterlassene Nachrichten (auto-erstellt)
└── whatsapp_log.txt             ← Gesprächsprotokoll (auto-erstellt)
```

5.2 Kritische Pflicht-Dateien

Diese Dateien MÜSSEN vorhanden sein – das System startet sonst nicht:

Datei	Warum kritisch
kernel.py	Zentrales Gehirn – ohne ihn läuft nichts
providers.py	KI-Provider-Anbindung – wird von kernel.py importiert
skill_manager.py	Skill-Loader – ohne ihn kann Ilija keine Skills nutzen
agent_state.py	Zustandsverwaltung – wird überall importiert
autonomy_loop.py	Autonomie-Logik – wird von web_server & telegram_bot benötigt
skill_policy.py	Sicherheits-Layer – wird von autonomy_loop benötigt
skill_registry.py	Skill-Schutz – wird von kernel.py benötigt
templates/index.html	Web-Interface – ohne Template startet Flask nicht
static/app.js	Frontend-Logik – Seite wäre ohne JS funktionslos
static/style.css	Design – wird von index.html geladen
skills/__init__.py	Python-Package-Marker – ohne ihn können Skills nicht importiert werden
.env	API-Keys – ohne Keys kein Cloud-Provider

5.3 Auto-generierte Dateien

Diese Dateien/Ordner werden automatisch beim ersten Start erstellt. Du musst sie nicht manuell anlegen:

- memory/ilija_db/ – ChromaDB-Datenbank für das Langzeitgedächtnis
- whatsapp_kalender.txt – Terminkalender (inkl. Verfügbarkeits-Vorlage)
- whatsapp_nachrichten.txt – Datei für hinterlassene Nachrichten
- whatsapp_log.txt – Gesprächsprotokoll aller WhatsApp-Chats
- skill_scores.json – Skill-Statistiken und Bewertungen

5.4 Arbeitsverzeichnis

⚠️ **Wichtig: Immer aus dem Projektroot starten!**
Alle Python-Dateien importieren sich gegenseitig relativ.
Du MUSST immer aus dem Ordner starten, wo kernel.py, web_server.py usw. liegen.
Falsch: cd / && python /pfad/zu/Ilija_full_evo2/web_server.py
Richtig: cd /pfad/zu/Ilija_full_evo2 && python web_server.py

KAPITEL 6

Bedienung

Alle Interfaces und Befehle

6.1 Ilija starten

Vor jedem Start: Virtuelle Umgebung aktivieren und in den Projektordner wechseln.

```
cd /pfad/zu/Ilija_full_evo2
source venv/bin/activate
```

Interface	Startbefehl	Erreichbar unter
Web-Interface	python web_server.py	http://localhost:5000
Telegram-Bot	python telegram_bot.py	Telegram-App → dein Bot
Beide parallel	python telegram_bot.py & python web_server.py	Beides gleichzeitig
Terminal-Modus	python kernel.py	Direkt in der Konsole
Terminal + Provider	python kernel.py --provider claude	Mit bestimmtem Provider

6.2 Web-Interface

Das Web-Interface ist die empfohlene Bedienoberfläche. Es bietet:

- Chat mit Ilija per Text
- Mikrofon-Aufnahme: Klick auf das Mikrofon-Symbol, Aufnahme starten, nochmal klicken zum Stoppen
- Datei-Upload: Klick auf die Büroklammer, Datei auswählen (PDF, DOCX, TXT, Code...)
- Skill-Reload-Button: Neu erstellte oder veränderte Skills sofort laden (↻)
- Provider-Wechsel: KI-Modell im Dropdown oben rechts wechseln
- Intent-Badges: Zeigen welche Absicht Ilija erkannt hat
- Skill-Badges: Zeigen welcher Skill ausgeführt wurde

💡 Mikrofon auf HTTP vs. HTTPS

Auf HTTPS oder localhost: Live-Aufnahme direkt im Browser.
Auf HTTP (lokales Netzwerk via IP): Automatisch Fallback auf Diktiergerät.
Android: 'Diktiergerät verwenden' auswählen – Aufnahme direkt von dort.
Für echte Live-Aufnahme über IP: HTTPS-Zertifikat einrichten.

6.3 Telegram-Bot-Befehle

Befehl	Funktion
/start	Bot starten, Willkommensnachricht
/help	Alle verfügbaren Befehle anzeigen
/reload	Skills neu laden (nach Skill-Erstellung)
/status	System-Status anzeigen (Provider, Skills, ...)
/clear	Chat-Verlauf zurücksetzen
Textnachricht	Direkt an Ilija – wie im Web-Interface
Sprachnachricht	Wird automatisch transkribiert (Whisper) und verarbeitet
Bild senden	Ilija analysiert das Bild (Claude Vision oder GPT-4o)
Datei senden	PDF, DOCX, TXT, Code – wird extrahiert und analysiert

6.4 Terminal-Modus-Befehle

Befehl	Funktion
reload	Skills neu laden
debug	System-Status anzeigen (Provider, Skills, Fehler)
clear	Chat-Verlauf löschen
switch	KI-Provider wechseln (claude/gpt/gemini/ollama)
exit / quit	Programm beenden
(beliebiger Text)	Nachricht an Ilija

6.5 Beispiel-Befehle an Ilija

Befehl	Was passiert
"Wer bist du?"	Ilija stellt sich vor
"Merke dir: Ich heiße Manuel"	Wird dauerhaft ins Gedächtnis gespeichert
"Was weißt du über mich?"	Ruft alle Erinnerungen über den Nutzer ab
"Erstelle einen Skill für X"	Ilija schreibt Python-Code, speichert und lädt ihn
"Überwache alle WhatsApp-Chats"	Startet den WhatsApp-Listener (Modus: alle)
"Starte WhatsApp-Anrufbeantworter"	Anrufbeantworter-Modus für WhatsApp
"Überwache Kontakt: Anna"	Überwacht nur den Kontakt 'Anna'
"Zeig mir den WhatsApp-Kalender"	Zeigt alle eingetragenen Termine
"Zeig mir hinterlassene Nachrichten"	Zeigt alle WhatsApp-Nachrichten
"Stoppe den WhatsApp-Listener"	Beendet den WhatsApp-Hintergrundprozess

Befehl	Was passiert
"Lies diese Webseite: [URL]"	Ilija liest und fasst die Webseite zusammen
"Würfle eine Zahl zwischen 1 und 100"	Nutzt den Würfel-Skill

KAPITEL 7

WhatsApp-Assistent

Autonomer Dialog, Kalender & Nachrichten

7.1 Voraussetzungen

- Google Chrome muss installiert sein
- WhatsApp Web muss im Chrome einmalig eingeloggt sein (QR-Code scannen)
- Der Skill benötigt einen laufenden Chrome-Browser im Hintergrund

7.2 WhatsApp Modi

Modus	Befehl	Beschreibung
kontakt	"Überwache Kontakt: [Name]"	Überwacht und beantwortet nur einen bestimmten Kontakt
alle	"Überwache alle WhatsApp-Chats"	Überwacht alle ungelesenen Nachrichten und antwortet
anrufbeantworter	"Starte WhatsApp-Anrufbeantworter"	Stellt sich als KI-Assistent vor, nimmt Nachrichten an

7.3 Kalender-Funktion

Der WhatsApp-Assistent kann eigenständig Termine vereinbaren. Die Verfügbarkeit wird in der Datei whatsapp_kalender.txt verwaltet:

```
# Verfügbarkeit konfigurieren (Datei: whatsapp_kalender.txt)
[VERFÜGBAR] [Montag-Freitag] [09:00-12:00]
[VERFÜGBAR] [Dienstag] [15:00-17:00]
[GESPERRT]  [Samstag-Sonntag]

# Eingetragene Termine (automatisch von Ilija ergänzt):
[2026-03-18] [Mittwoch] [10:00] [Kontakt-Name] Betreff
```

Ilija prüft bei jeder Terminanfrage ob der gewünschte Zeitslot frei ist. Doppelbuchungen werden automatisch verhindert.

7.4 Nachrichten-Funktion

Wenn ein Kontakt eine Nachricht hinterlassen möchte, speichert Ilija sie in whatsapp_nachrichten.txt. Diese Nachrichten kannst du über Telegram oder das Web-Interface abrufen:

```
"Zeig mir hinterlassene Nachrichten"
```

KAPITEL 8

Skill-System

Eigene Fähigkeiten erstellen und verwalten

8.1 Eigene Skills erstellen

Ein Skill ist eine einfache Python-Datei im skills/-Ordner. Das Mindestformat:

```
# skills/mein_skill.py

def mein_skill(parameter: str) -> str:
    """
    Kurze Beschreibung fuer die KI - wann soll dieser Skill genutzt werden?
    parameter: Beschreibung des Parameters
    """
    return f"Ergebnis: {parameter}"

AVAILABLE_SKILLS = [mein_skill]
```

Nach dem Speichern: Reload-Button im Web-Interface klicken oder /reload in Telegram eingeben.

8.2 Von Ilija Skill erstellen lassen

Du musst Skills nicht selbst schreiben. Beschreibe Ilija einfach was du brauchst:

```
"Erstelle einen Skill der mir das aktuelle Wetter in Berlin anzeigt"
```

Ilija schreibt den Code, speichert ihn in skills/ und lädt ihn sofort. Der Skill ist danach dauerhaft verfügbar.

8.3 Skill-Policy (Sicherheit)

Policy-Stufe	Beschreibung	Beispiele
SAFE	Wird automatisch ausgeführt	Wetter, Würfeln, Gedächtnis lesen
INTERACTIVE	Ilija fragt nach Bestätigung	Webseiten laden, Dateien lesen
RISKY	Erfordert explizite Genehmigung	Shell-Befehle, Dateien schreiben

8.4 Mitgelieferte Skills

Skill-Datei	Funktionen
basis_tools.py	Aktuelle Zeit, Shell-Befehle, Datum
gedaechtnis.py	Langzeitgedächtnis speichern und abrufen (ChromaDB)
skill_factory_improved.py	Neue Skills zur Laufzeit erstellen
whatsapp_autonomer_dialog.py	WhatsApp-Überwachung, Terminbuchung, Nachrichten
whatsapp_lesen.py	WhatsApp-Nachrichten lesen
whatsapp_senden.py	WhatsApp-Nachricht senden
webseiten_inhalt_lesen.py	Webseiten lesen und zusammenfassen
browser_oeffnen.py	Browser steuern (Selenium)
datei_lesen.py	Dateien vom Dateisystem lesen
cmd_ausfuehren.py	Shell-Befehle ausführen
outlook_posteingang_pruefen.py	Outlook-Posteingang lesen
trading.py	Trading-Informationen abrufen
wetter_offenburg_abfragen.py	Wetter für Offenburg (Beispiel-Skill)
muenze_werfen.py	Münze werfen (Zufall)
wuerfeln.py	Würfeln mit verschiedenen Seiten
witze_erzellen.py	Witze generieren
wissen_bearbeiten.py	Wissen strukturiert verwalten

KAPITEL 9

Fehlerbehandlung

Diagnose und Lösungen für häufige Probleme

9.1 Diagnose-Tools

Bevor du einen Fehler analysierst: Überprüfe zuerst den System-Status.

```
# Terminal-Modus starten:
python kernel.py

# debug-Befehl eingeben:
debug

# Zeigt:
# - Aktiver Provider
# - Anzahl geladener Skills
# - Letzte Fehler
# - Gedächtnis-Status
```

```
# Logs in Echtzeit beobachten:
tail -f offenes_leuchten.log
tail -f web_server.log

# Python-Imports testen:
source venv/bin/activate
python -c "from kernel import Kernel; print('OK')"
```

9.2 Installations-Fehler

FEHLER: Python-Version zu alt

```
# Fehlermeldung:
Python 3.8 gefunden - mindestens 3.10 benoetigt!

# Loesung:
sudo apt update
sudo apt install python3.10 python3.10-venv python3.10-pip

# Dann mit expliziter Version:
python3.10 -m venv venv
```

FEHLER: pip install schlägt fehl

```
# Fehlermeldung:
error: externally-managed-environment

# Loesung: Virtuelle Umgebung nutzen!
python3 -m venv venv
source venv/bin/activate
pip install ...
```

```
# Fehlermeldung:
ERROR: Could not build wheels for chromadb

# Loesung:
sudo apt install python3-dev build-essential
pip install --upgrade pip setuptools wheel
pip install chromadb
```

FEHLER: sentence-transformers Download schlägt fehl

```
# Fehlermeldung:
OSError: [Errno 28] No space left on device

# Loesung: Speicherplatz pruefen:
df -h

# Modell-Cache ist standardmaessig in ~/.cache/huggingface
# Anderen Pfad setzen:
export TRANSFORMERS_CACHE=/pfad/mit/mehr/platz
```

9.3 Start-Fehler

FEHLER: ModuleNotFoundError

```
# Fehlermeldung:
ModuleNotFoundError: No module named 'flask'

# Ursache: Virtuelle Umgebung nicht aktiviert!

# Loesung:
source venv/bin/activate
python web_server.py
```

```
# Fehlermeldung:
ModuleNotFoundError: No module named 'bs4'

# Ursache: beautifulsoup4 fehlt
```

```
# Loesung:
pip install beautifulsoup4 lxml
```

```
# Fehlermeldung:
ModuleNotFoundError: No module named 'skill_manager'

# Ursache: Falsches Arbeitsverzeichnis!

# Loesung: In den Projektordner wechseln:
cd /pfad/zu/Ilija_full_evo2
python web_server.py
```

FEHLER: Port bereits belegt

```
# Fehlermeldung:
OSError: [Errno 98] Address already in use

# Loesung 1: Anderen Port verwenden:
# In web_server.py: app.run(port=5001)

# Loesung 2: Prozess der Port 5000 belegt finden und beenden:
sudo lsof -i :5000
kill -9 <PID>
```

9.4 KI-Provider-Fehler

FEHLER: Kein Provider verfügbar

```
# Fehlermeldung:
ProviderError: Kein Provider verfuegbar

# Checkliste:
# 1. .env-Datei vorhanden?
ls -la .env

# 2. API-Key eingetragen?
cat .env

# 3. Ollama installiert und laufend?
ollama list
ollama serve  # Falls Ollama nicht laeuft
```

FEHLER: API-Authentifizierung schlägt fehl

```
# Fehlermeldung (Claude):
anthropic.AuthenticationError: invalid x-api-key
```

```
# Fehlermeldung (OpenAI):
openai.AuthenticationError: Incorrect API key

# Loesung: API-Key pruefen
# 1. .env oeffnen: nano .env
# 2. Key hat keine Leerzeichen oder Anfuehrungszeichen
# 3. Key ist noch gueltig (Konsole des Providers pruefen)
# 4. .env neu laden:
# Programm neu starten
```

FEHLER: Rate Limit

```
# Fehlermeldung:
RateLimitError: Rate limit exceeded

# Loesung: Ilija wechselt automatisch zum naechsten Provider.
# Wenn alle Limits erschoepft: Ollama als Fallback nutzen.

# Oder: Anderen Provider manuell setzen:
# Im Web-Interface: Provider-Dropdown oben rechts
# Im Terminal: switch eingeben
```

9.5 WhatsApp-Fehler

FEHLER: Chrome/Selenium startet nicht

```
# Fehlermeldung:
WebDriverException: 'chromedriver' executable needs to be in PATH

# Loesung: webdriver-manager installieren (automatisch ChromeDriver)
pip install webdriver-manager

# Fehlermeldung:
Message: session not created: Chrome version must be ...

# Loesung: Chrome aktualisieren
sudo apt upgrade google-chrome-stable
```

```
# Fehlermeldung:
error: cannot connect to chrome at 127.0.0.1

# Loesung: Headless-Modus bei Server ohne Display:
# Xvfb installieren:
sudo apt install xvfb
Xvfb :99 -screen 0 1920x1080x24 &
export DISPLAY=:99
python web_server.py
```

FEHLER: WhatsApp Web – QR-Code wird nicht erkannt

- Chrome muss geöffnet sein und WhatsApp Web eingeloggt sein (Sitzung aktiv).
- WhatsApp-Sitzung ist abgelaufen: Manuell in Chrome einloggen und QR-Code erneut scannen.
- Chrome-Profil-Pfad prüfen: Der Skill merkt sich die Sitzung im Chrome-Profil.

FEHLER: Kontakt nicht gefunden

```
# Fehler: Kontakt 'Anna' nicht in der Kontaktliste gefunden

# Loesung:
# 1. Kontaktname genau wie in WhatsApp angeben
# 2. Gross-/Kleinschreibung beachten
# 3. Telefonnummer statt Name versuchen
```

9.6 Telegram-Fehler

FEHLER: Bot antwortet nicht

```
# Checkliste:
# 1. Telegram-Bot laeuft?
ps aux | grep telegram_bot

# 2. Bot-Token korrekt?
grep TELEGRAM_BOT_TOKEN .env

# 3. Deine User-ID in der Whitelist?
grep TELEGRAM_ALLOWED_USERS .env

# 4. Logs pruefen:
tail -f offenes_leuchten.log
```

9.7 Gedächtnis-Fehler

FEHLER: ChromaDB schlägt fehl

```
# Fehlermeldung:
chromadb.errors.InvalidCollectionException

# Loesung: Gedaechtnis-Datenbank zurueckcksetzen (Erinnerungen gehen verloren!):
rm -rf memory/ilija_db
mkdir -p memory
python web_server.py  # Datenbank wird neu erstellt
```

```
# Fehlermeldung:
RuntimeError: ONNX Runtime error

# Loesung:
pip install --upgrade onnxruntime
pip install --upgrade sentence-transformers
```

9.8 Web-Interface-Fehler

FEHLER: Seite lädt nicht

```
# Pruefen ob der Server laeuft:
curl http://localhost:5000

# Falls Fehler 500:
python web_server.py  # Fehlermeldung im Terminal lesen

# Falls Seite leer:
# Browser-Cache leeren: Strg+Shift+R
```

FEHLER: Mikrofon-Aufnahme funktioniert nicht

- HTTP (nicht HTTPS): Automatisch Fallback auf Diktiergerät – das ist normal und gewollt.
- HTTPS lokal einrichten: flask-talisman oder nginx als Reverse-Proxy mit selbst-signiertem Zertifikat.
- Browser-Berechtigung: Im Browser-Adressbalken → Schloss-Symbol → Mikrofon erlauben.

9.9 Vollständige Neuinstallation

Wenn nichts mehr hilft: saubere Neuinstallation ohne Datenverlust.

```
# 1. Virtuelle Umgebung loeschen:
rm -rf venv

# 2. Python-Caches loeschen:
find . -type d -name __pycache__ -exec rm -rf {} + 2>/dev/null
find . -name '*.pyc' -delete

# 3. (Optional) Gedaechtnis zuruecksetzen:
# rm -rf memory/ilija_db

# 4. Frisch installieren:
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
```

```
pip install -r requirements.txt

# 5. Neu starten:
python web_server.py
```

⚠️ **Daten sichern vor dem Zurücksetzen**

memory/ enthält dein Langzeitgedächtnis – bei rm -rf memory/ilija_db geht es verloren.
whatsapp_kalender.txt und whatsapp_nachrichten.txt vor Reset sichern.
.env sichern – enthält deine API-Keys.
skills/ mit eigenen Skills sichern.

KAPITEL 10

Häufige Fragen (FAQ)

Antworten auf häufig gestellte Fragen

Welcher Provider ist am besten?

Claude (Anthropic) liefert die beste Qualität und versteht komplexe Aufgaben am besten. Gemini (Google) ist kostenlos für moderaten Einsatz. Ollama ist ideal wenn du offline arbeiten möchtest oder Datenschutz wichtig ist.

Ilija vergisst alles nach dem Neustart

Das passiert wenn ChromaDB nicht korrekt installiert ist oder das Memory-Verzeichnis nicht beschreibbar ist. Überprüfe ob memory/ existiert und ob die Datenbank in memory/ilija_db/ angelegt wurde.

Kann ich Ilija auf einem Server im Dauerbetrieb laufen lassen?

```
# Mit systemd als Dienst einrichten:
sudo nano /etc/systemd/system/ilija.service

[Unit]
Description=Ilija KI-Agent
After=network.target

[Service]
WorkingDirectory=/pfad/zu/Ilija_full_evo2
ExecStart=/pfad/zu/Ilija_full_evo2/venv/bin/python web_server.py
Restart=always
User=deinnutzer

[Install]
WantedBy=multi-user.target

# Dienst aktivieren:
sudo systemctl enable ilija
sudo systemctl start ilija
```

Wie füge ich einen neuen Skill hinzu?

Python-Datei mit AVAILABLE_SKILLS-Liste in den skills/-Ordner legen und danach den Reload-Button klicken oder /reload in Telegram tippen. Oder: Ilija direkt bitten den Skill zu erstellen.

Ist das System sicher? Kann jemand anderes meinen Telegram-Bot steuern?

Nein – dank TELEGRAM_ALLOWED_USERS ist der Zugriff auf deine User-ID beschränkt. Alle anderen Nutzer erhalten keine Antwort. WhatsApp-Kontakte können keine Kernel-Befehle ausführen – sie haben nur Zugriff auf den Dialog-Modus.

Kann ich Ilija auf Windows nutzen?

Grundsätzlich ja, aber mit Einschränkungen. Das install.sh-Skript läuft nur auf Linux/macOS. Unter Windows muss die manuelle Installation (Kapitel 3) verwendet werden. Der WhatsApp-Skill und einige Shell-Skills sind Linux-optimiert. WSL2 (Windows Subsystem for Linux) unter Windows wird empfohlen.

Offenes Leuchten v5.0 – Ilija

MIT License – Freie Nutzung, Weitergabe und Modifikation

GitHub: github.com/<user>/offenes-leuchten